

```
import numpy as np
from sklearn.datasets import load_iris, load_wine, fetch_california_housing
from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.metrics import accuracy_score, f1_score, mean_squared_error
```

```
class SimpleDecisionTree:
    def __init__(self, max_depth=3):
        self.max_depth = max_depth

    def fit(self, X, y):
        self.tree = self._build(X, y, 0)

    def _entropy(self, y):
        probs = np.bincount(y) / len(y)
        return -np.sum(probs * np.log2(probs + 1e-9))

    def _build(self, X, y, depth):
        if len(np.unique(y)) == 1 or depth == self.max_depth:
            return np.bincount(y).argmax()

        best_gain = 0
        best_feature, best_threshold = None, None

        for f in range(X.shape[1]):
            for t in np.unique(X[:, f]):
                left = y[X[:, f] <= t]
                right = y[X[:, f] > t]
                if len(left) == 0 or len(right) == 0:
                    continue

                gain = self._entropy(y) - (
                    len(left)/len(y)*self._entropy(left) +
                    len(right)/len(y)*self._entropy(right)
                )

                if gain > best_gain:
                    best_gain, best_feature, best_threshold = gain, f, t

        if best_feature is None:
            return np.bincount(y).argmax()

        left_idx = X[:, best_feature] <= best_threshold
        right_idx = X[:, best_feature] > best_threshold

        return (best_feature, best_threshold,
                self._build(X[left_idx], y[left_idx], depth+1),
                self._build(X[right_idx], y[right_idx], depth+1))

    def predict(self, X):
        return [self._predict_one(x, self.tree) for x in X]

    def _predict_one(self, x, tree):
        if not isinstance(tree, tuple):
            return tree
        f, t, left, right = tree
        return self._predict_one(x, left if x[f] <= t else right)
```

```
iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
custom_tree = SimpleDecisionTree()
custom_tree.fit(X_train, y_train)

y_pred_custom = custom_tree.predict(X_test)
```

```
print("Custom Decision Tree Accuracy:",
      accuracy_score(y_test, y_pred_custom))
```

Custom Decision Tree Accuracy: 1.0

```
sk_tree = DecisionTreeClassifier(max_depth=3, random_state=42)
sk_tree.fit(X_train, y_train)

y_pred_sk = sk_tree.predict(X_test)
print("Scikit-learn Decision Tree Accuracy:",
      accuracy_score(y_test, y_pred_sk))
```

Scikit-learn Decision Tree Accuracy: 1.0

```
print("\nAccuracy Comparison")
print("Custom Tree      :", accuracy_score(y_test, y_pred_custom))
print("Scikit-learn DT :", accuracy_score(y_test, y_pred_sk))
```

Accuracy Comparison
 Custom Tree : 1.0
 Scikit-learn DT : 1.0

PART 2: Ensemble Methods & Hyperparameter Tuning (Wine Dataset)

```
wine = load_wine()
X, y = wine.data, wine.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
dt = DecisionTreeClassifier(random_state=42)
rf = RandomForestClassifier(random_state=42)

dt.fit(X_train, y_train)
rf.fit(X_train, y_train)

print("Decision Tree F1:",
      f1_score(y_test, dt.predict(X_test), average='weighted'))

print("Random Forest F1:",
      f1_score(y_test, rf.predict(X_test), average='weighted'))
```

Decision Tree F1: 0.9439974457215836
 Random Forest F1: 1.0

```
param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [None, 5],
    'min_samples_split': [2, 5]
}

grid = GridSearchCV(RandomForestClassifier(), param_grid, cv=3)
grid.fit(X_train, y_train)

print("Best RF Parameters:", grid.best_params_)
```

Best RF Parameters: {'max_depth': 5, 'min_samples_split': 2, 'n_estimators': 50}

PART 3: Regression + Hyperparameter Tuning

```
housing = fetch_california_housing()
X, y = housing.data, housing.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
```

```
)
```

```
dt_reg = DecisionTreeRegressor(random_state=42)
rf_reg = RandomForestRegressor(random_state=42)

dt_reg.fit(X_train, y_train)
rf_reg.fit(X_train, y_train)

print("DT MSE:",
      mean_squared_error(y_test, dt_reg.predict(X_test)))

print("RF MSE:",
      mean_squared_error(y_test, rf_reg.predict(X_test)))
```

```
DT MSE: 0.495235205629094
RF MSE: 0.2553684927247781
```

```
param_dist = {
    'n_estimators': [50, 100],
    'max_depth': [None, 10],
    'min_samples_split': [2, 5]
}

random_search = RandomizedSearchCV(
    RandomForestRegressor(),
    param_dist,
    n_iter=3,
    cv=3
)

random_search.fit(X_train, y_train)
print("Best Regression Parameters:", random_search.best_params_)
```

```
Best Regression Parameters: {'n_estimators': 100, 'min_samples_split': 2, 'max_depth': None}
```